

● ARCHITECTURE WALKTHROUGH — DEMO EDITION

Platform Architecture Deep-Dive

A ground-up explanation of the repository's structure, contracts, and design patterns — for architecture review. The pattern is domain-agnostic; a concrete reference implementation grounds every section.

DOC NO.	STATUS	AUDIENCE	FORMAT
ARCH-DEMO-01	For Review	Architecture Board	Live Walkthrough

01 – Folder structure 02 – Contract layer 03 – REST entry layer 04 – Shared lib

01 Folder structure, top to bottom

```
platform/
├─ pom.xml                ← Maven parent: module list + shared dependency ve
├─ docker-compose.yml     ← one-command local infra (§10)
├─
├─ common-library/        ← shared jar (§04)
├─ protos/                ← shared gRPC contracts (§02)
├─
├─ <service-a>/           ← one Maven module = one deployable
├─ <service-b>/
├─ <service-c>/           (repeat per bounded context)
├─
├─ architecture-tests/    ← ArchUnit: enforces layering at build time (§12)
├─
├─ helm/<service>/        ← one Helm chart per service (§07)
├─ infra/                 ← local-dev-only provisioning (§08)
├─ scripts/               ← thin operational wrappers (§08)
├─ docs/                  ← reference documentation (§08)
```

Every top-level folder answers exactly one question about the system: *what's the shared contract* (`protos`), *what's genuinely shared code* (`common-library`), *what runs as its own process* (each service folder), *does the layering stay honest* (`architecture-tests`), *how does this deploy* (`helm`), *how do I run this locally* (`infra + docker-compose.yml + scripts`). No folder answers two of those questions at once.

DOES EVERY SERVICE NEED ITS OWN DATABASE?

No — "one service, one database" applies to services that own persistent state, not a mandate that every module must have one. A stateless service (a router, an aggregator, a notification dispatcher) is still its own Maven module and deployable, but simply has no `entity/`, `repository/`, or `db/migration/` — no JPA/Flyway dependency, no datasource config. ArchUnit's entity/repository rules (§12) don't apply to it because those packages don't exist there.

IF SOME STATE IS STILL NEEDED

A short-lived idempotency cache or rate-limit counter still gets **its own store** (e.g. Redis), scoped to that one service — never borrowed from another service's database.

02 The contract layer — `protos/`

```
protos/src/main/proto/
├── common/v1/
│   └── common.proto           ← shared message types
├── <context-a>/v1/
│   └── <context_a>_service.proto
├── <context-b>/v1/
│   └── <context_b>_service.proto
└── ...                       ← one folder per bounded context
```

One package per bounded context, each independently versioned (`v1`, `v2`, ...). If one context's contract needs a breaking change, only that context's `v2/` gets created

— every other service is unaffected. A single shared proto file would force a coordinated version bump across the whole system for any change.

WHY ITS OWN MAVEN MODULE, NOT COPIED INTO EACH SERVICE

`protos/pom.xml` runs the `protobuf-maven-plugin`, invoking `protoc` and `protoc-gen-grpc-java` to turn every `.proto` file into real Java classes — message classes, and a generated service class containing both the server-side base class and the client-side stub. That generation happens once; the result is installed as one jar, and every consuming service adds a single dependency line and gets byte-for-byte the same generated classes. No service maintains its own copy of another service's contract.

03 The REST entry layer

Every externally callable service has a `controller/` package: thin classes whose only job is translating HTTP concepts into a plain method call.

```
@RestController
@RequestMapping("/api/v1/uploads")
public class UploadController {
    @PostMapping("/initiate")
    public ResponseEntity<InitiateUploadResponse> initiate(@Valid @RequestBody InitiateUploadRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(uploadService.initiate(request));
    }
}
```

What this layer does: deserializes the request body, validates it before the method body runs, extracts path/query values, picks the right status code. **What it explicitly never does:** business logic, direct repository access, direct infrastructure calls — all of that is one line away, in `service/`. Enforced, not just convention — see §12.

Paired with a `GlobalExceptionHandler` per service that turns domain exceptions into a consistent JSON error shape with the right status code — the controller method itself never contains a `try/catch`.

04 The shared library

`common-library` is a plain Maven dependency, not a running service. It holds exactly two kinds of things:

- **Cross-cutting shapes and base types** used identically everywhere — a shared error-response shape, a small exception base hierarchy, small utilities.
- **A shared infrastructure client with its own auto-wiring** — an object-storage client is the concrete example here (§06), but the mechanism applies to any cross-cutting infrastructure concern.

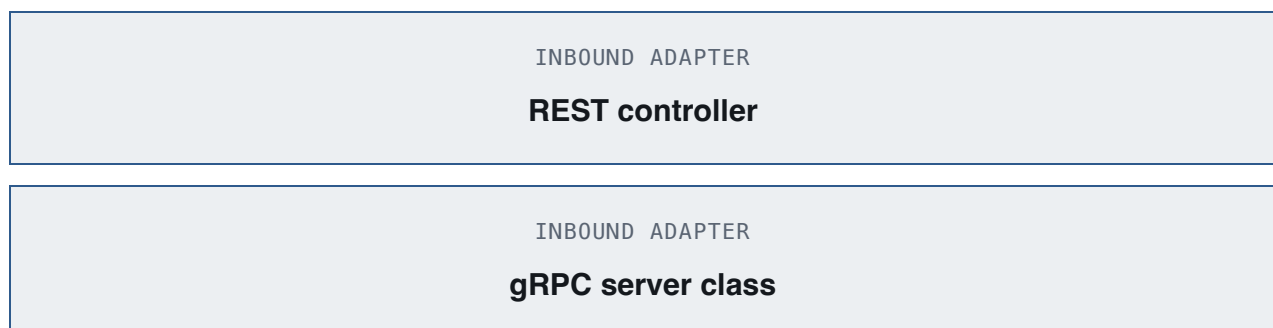
THE DISCIPLINE THAT KEEPS THIS SAFE

Nothing service-specific goes in here — no entity definitions, no service-specific DTOs, no business rules. A "shared" domain class here would ripple across services meant to deploy independently, quietly recreating a distributed monolith under the appearance of microservices.

05 Hexagonal architecture, without a multi-module split

Packages are flat

(`controller/dto/entity/exception/mapper/repository/service`) rather than split into `domain/application/infrastructure` modules — but the hexagonal principle (business logic depends only on interfaces it owns; infrastructure depends on business logic, never the reverse) is fully in force, enforced by package convention plus ArchUnit (§12) instead of module boundaries.



INBOUND ADAPTER

@KafkaListener

SERVICE/ — THE CORE

business logic

TALKS ONLY TO INTERFACES IT OWNS

OUTBOUND ADAPTER

JPA repository

OUTBOUND ADAPTER

ObjectStorageClient impl

OUTBOUND ADAPTER

KafkaTemplate producer

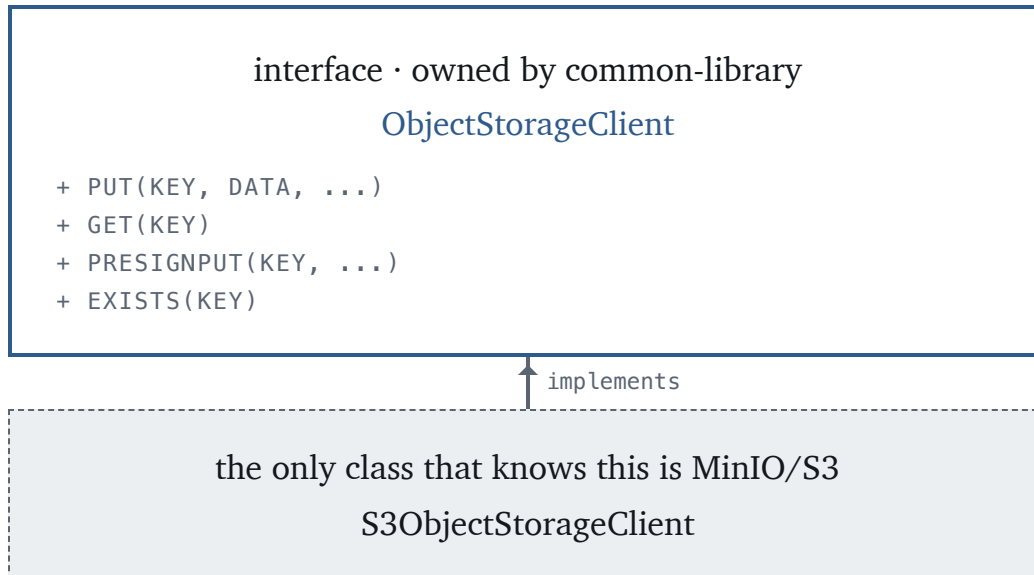
entity/ — the domain model itself, sitting inside the core

THE DEPENDENCY RULE THAT MATTERS

service/ classes import repository/ interfaces and common-library's storage interface — never a concrete Hibernate class beyond what Spring Data generates, never a concrete SDK class. Infrastructure depends inward on the business logic's interfaces; the business logic never depends outward on a specific infrastructure choice.

06 Strategy pattern & dependency inversion: swappable infrastructure

The concrete mechanism that makes "change the storage backend without touching business logic" actually true, not just an aspiration.



Business logic is constructed with the *interface* as a constructor parameter and calls `objectStorageClient.presignPut(...)` with no idea MinIO is on the other end — it could be AWS S3, GCS, Azure Blob Storage, or a local filesystem adapter for tests. Swapping the backend means writing one new class implementing the interface; the business logic doesn't change by a single line.

Spring's dependency injection is *how* the swap happens without a factory or a stray new anywhere: `common-library` ships an auto-configuration that builds the concrete client and exposes it as the interface's bean, gated so it only activates where it's actually configured. A second implementation would be a configuration-time decision, never a code change.

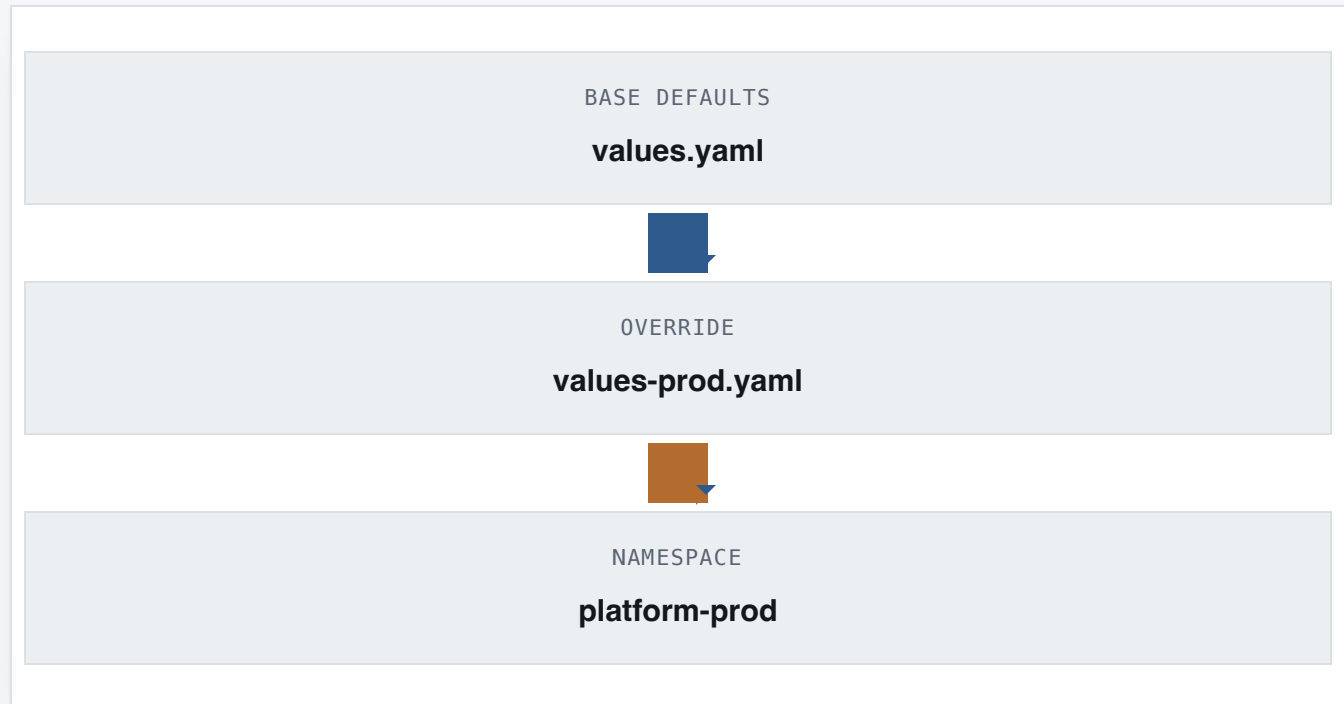
THE SAME PATTERN, A SECOND SEAM

A file-processing service defines a `FileProcessorStrategy` interface with one implementation per file type. Spring collects every implementation into a list; a small registry picks the right one at runtime. Onboarding a new file type is "write one new class" — zero changes to the code that uses it.

07 Deployment — how Helm fits

One Helm chart per service, because each service scales, deploys, and rolls back **independently** — the entire point of the one-module-one-deployable structure (§01). `values.yaml` per service captures what differs per environment: replica count, resource limits, image tag, autoscaling thresholds.

MULTIPLE ENVIRONMENTS — THE CHART DOESN'T CHANGE, THE VALUES DO



```
helm upgrade --install upload-service helm/upload-service \
  -f helm/upload-service/values.yaml \
  -f helm/upload-service/values-prod.yaml \
  --namespace platform-prod
```

Paired with one Kubernetes namespace per environment, so the same release name exists independently in each without colliding. This is a small extension of the existing deploy script (already parameterized by namespace) — not a restructure of the chart itself.

08 Scripts, docs, and local-dev infra

scripts/

docs/

Thin wrappers — nothing inside them isn't a plain `mvn`, `docker`, or `helm` command underneath. Their purpose is muscle-memory, not logic.

Each doc answers exactly one question — running locally, one service's code, a troubleshooting log, this architecture explanation — and links to its siblings instead of repeating them.

infra/

Local-dev-only provisioning (e.g. creating one database per service in a fresh container). No equivalent in a deployed environment, where each database already exists.

09 Configuration management

Every service reads configuration the same way — `application.yml` bound directly onto typed Java fields, no manual parsing anywhere:

```
insight:
  storage:
    endpoint: http://localhost:9000
    bucket: platform-uploads
```

Locally, `application.yml` holds real values pointing at the local Compose environment (§10). In a deployed environment, the same key names get overridden by environment variables or mounted config injected via the Helm chart (§07) — the YAML defaults are just defaults, not the source of truth in production. Secrets are placeholder plaintext values only locally; in any real environment they come from a secrets manager, injected the same way as any other override, never committed as real credentials.

10 What the local environment buys

One command reproduces every piece of infrastructure every service needs locally, without any engineer hand-installing or hand-configuring it.

Onboarding speed

One command instead of a multi-page setup guide for a message broker, databases, and object storage individually.

Environment parity

The same broker/database/storage *technology* runs locally as in a real environment — integration bugs surface locally, not only in a shared environment.

Disposability

Tearing it down and back up starts from a guaranteed-clean slate — valuable because local init scripts only run once, against an empty volume.

11 The outbox pattern

The problem it solves: a service's database and a message broker are two different systems — nothing makes "commit this row" and "publish this message" atomic for free. Publish-then-commit risks a phantom message if the transaction rolls back; commit-then-publish risks a silently lost one if the publish fails.

ONE DATABASE TRANSACTION

1. INSERT the domain row (e.g. mark a submission accepted)
2. INSERT an outbox row: {topic, key, payload, published=false}



both committed together, or neither did



SEPARATE @SCHEDULED POLLER

runs every $\sim 1s$, independent of any request



`SELECT ... WHERE published = false`



publish to broker $\rightarrow$ `UPDATE published = true`

TRADE-OFF ACCEPTED

A small publish delay (bounded by the poller's interval) in exchange for never losing or fabricating an event relative to what's actually in the database.

12 Enforcing layering at build time

Because packages are flat (§01, §05) rather than split into separate Maven modules, nothing stops a careless change from having a controller call a repository directly — unless something checks for it. A small ArchUnit suite, run against the **compiled classes** of every service as part of the normal test phase, is that check:

```
@Test
void controllersMustNotAccessRepositoriesDirectly() {
    ArchRule rule = classes()
        .that().resideInAPackage("..controller..")
        .should().onlyAccessClassesThat().resideOutsideOfPackage("..repository..");
    rule.check(classes());
}
```

If someone adds a repository call directly inside a controller method tomorrow, this suite fails the build — not a review comment that can be missed, an actual red build. This is what lets the repository stay single-module-per-service (§05's trade-off) without that convenience quietly eroding over time.

13 Putting it together — the saga, end to end

1. A service accepts a unit of work and, in one transaction, persists it *and* writes an outbox row (§11). The relay publishes moments later.
2. An **orchestrator** consumes that event, creates its own saga record in its own database, and dispatches the first processing step via a synchronous, typed gRPC call.
3. Each processing service, on finishing (or failing), publishes its own completion event — again via its own outbox.
4. The orchestrator consumes each completion event, advances the saga's state machine, and dispatches the next step — conditionally skipping stages that don't apply — until the saga reaches a terminal state.

TRADE-OFFS ACCEPTED, EXPLICITLY

Latency vs. correctness

Every cross-service step incurs the outbox relay's polling delay on top of processing time — accepted because the alternative risks losing or fabricating saga-critical events.

Coordinator as critical path

The orchestrator's availability now gates the whole pipeline — accepted because centralized status visibility and compensation logic were requirements this pattern specifically solves for.

Eventual, not immediate, consistency

The orchestrator's view and a processing service's view may be momentarily out of sync — accepted because a distributed transaction isn't available once each service owns its own database.

Each of these is a deliberate choice with a named alternative and a stated reason it was rejected — not an oversight to be discovered later.

